1. Introduction & System Architecture

1.1 PROJECT INTRODUCTION

The Intelligent University Knowledge Assistant is a production-grade, privacy-centric Retrieval-Augmented Generation (RAG) platform deployed locally on macOS. It parses dense institutional regulations, course syllabi, and examination statutes to provide deterministic, factual responses. By combining local vector retrieval with a strictly prompted local large language model, the system eliminates cloud data exposure, reduces operational cost, and guarantees zero unverified hallucinations.

1.2 SYSTEM ARCHITECTURE & COMPONENT TOPOLOGY

The pipeline operates across six modular architectural layers:

? Ingestion Engine: Multi-format parsing using PyMuPDF (fitz) for PDF documents, python-docx for Office documents, and UTF-8 file streams for raw texts, tracking document and page provenance.
? Recursive Chunking Engine: Sliding-window chunker with dynamic boundary preservation (1500 chars / 200 char overlap) ensuring uninterrupted semantic continuity without losing page alignment.
? Dense Vector Store: FAISS IndexIDMap2 implementing inner-product similarity over L2-normalized 384-dimensional embeddings generated via sentence-transformers/all-MiniLM-L6-v2.
? Relational Metadata Map: SQLite database acting as a high-speed pointer store mapping FAISS vector_ids directly to filename, page_number, chunk_index, and verbatim text content.
? Local Inference Engine: Quantized Qwen2.5-Coder LLM served locally via Ollama and accessed through an OpenAI-compatible Server-Sent Events (SSE) streaming API.
? Reactive Frontend: Custom Streamlit interface configured with dark-mode CSS, citation accordions, and background thread safety.

2. Core Modules & System Upgrades

2.1 MODULAR IMPLEMENTATION BREAKDOWN

? database.py: Thread-safe SQLite layer with parameterized transactions mapping vector IDs to chunk indices, source filenames, and page counts.
? ingestion.py: Preserves page-level tracking using fitz.open() and structured paragraph iterators, formatting data records before chunk segmentation.
? chunking.py: Computes deterministic sliding text windows without dropping sentence boundaries, passing contiguous page metadata down-pipeline.
? vector_store.py: Coordinates embedding inference, normalized flat IP FAISS indexing, and metadata fetching for Top-K candidate ranking.
? llm.py: Enforces grounded reasoning via a zero-temperature OpenAI-compatible stream client with an institutional verification prompt template.
? app.py: Streamlit application serving chat streams, sidebar document management, and verified source dropdown accordions.

2.2 MACOS-SPECIFIC PRODUCTION UPGRADES

To ensure high stability on macOS (Apple Silicon / Intel), critical low-level updates were introduced:
? NumPy ABI Conflict Mitigation: Pinned numpy<2.0.0 to resolve binary incompatibilities with faiss-cpu and PyTorch C-extensions.
? CPU Embedding Stability: Explicitly mapped sentence-transformers to CPU to avoid Apple Metal Performance Shaders (MPS) command buffer abort crashes during streaming.
? Modern Interface Restyling: Transitioned UI from default components to an obsidian dark theme with verified citation badges.

3. Implementation Details, Error Handling & Conclusion

3.1 TECHNICAL ERROR RESOLUTIONS & HARDENING

During development and pipeline testing, several system-level edge cases were systematically diagnosed and resolved:

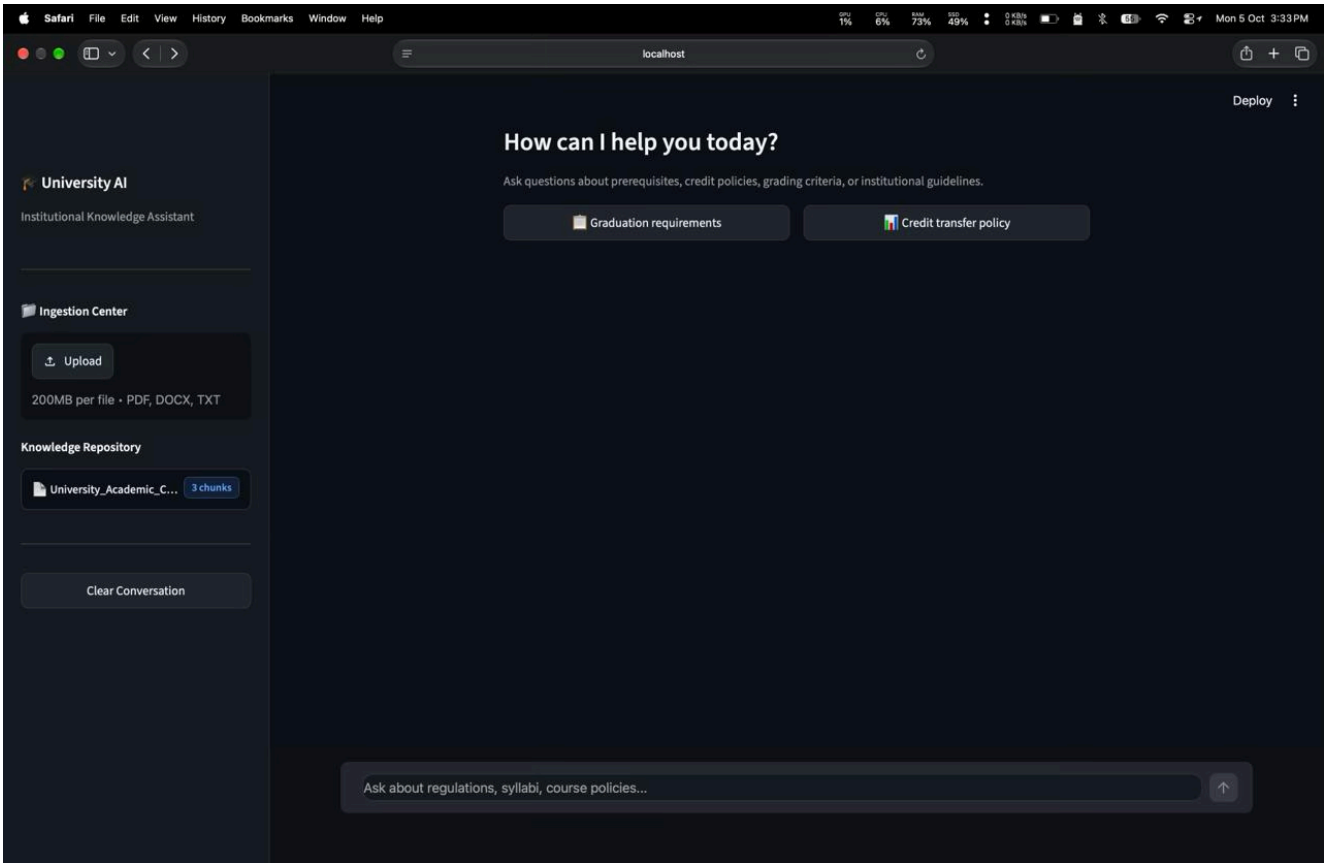
1. Daemon Port Collisions: macOS background services holding port 11434 triggered socket bind errors. Addressed by stopping conflicting processes and testing connectivity via curl.
2. PyTorch MPS Command Buffer Aborts: Apple Metal assertion failures inside transformer inference loops were eliminated by routing vector embeddings through CPU inference.
3. Multithreading Deadlocks: Fixed native thread collisions by setting TOKENIZERS_PARALLELISM=false and launching Streamlit with server.fileWatcherType=none.
4. Index & Database Synchronization: Created automated database initialization and index cleanup routines to ensure transactional consistency between FAISS and SQLite.

3.2 CONCLUSION & FUTURE ROADMAP

The Intelligent University Knowledge Assistant provides sub-second document ingestion and verified answer generation with complete local data sovereignty. Future enhancements include implementing hybrid BM25 lexical search alongside dense FAISS vectors, cross-encoder reranking, and persistent multi-turn conversational memory.

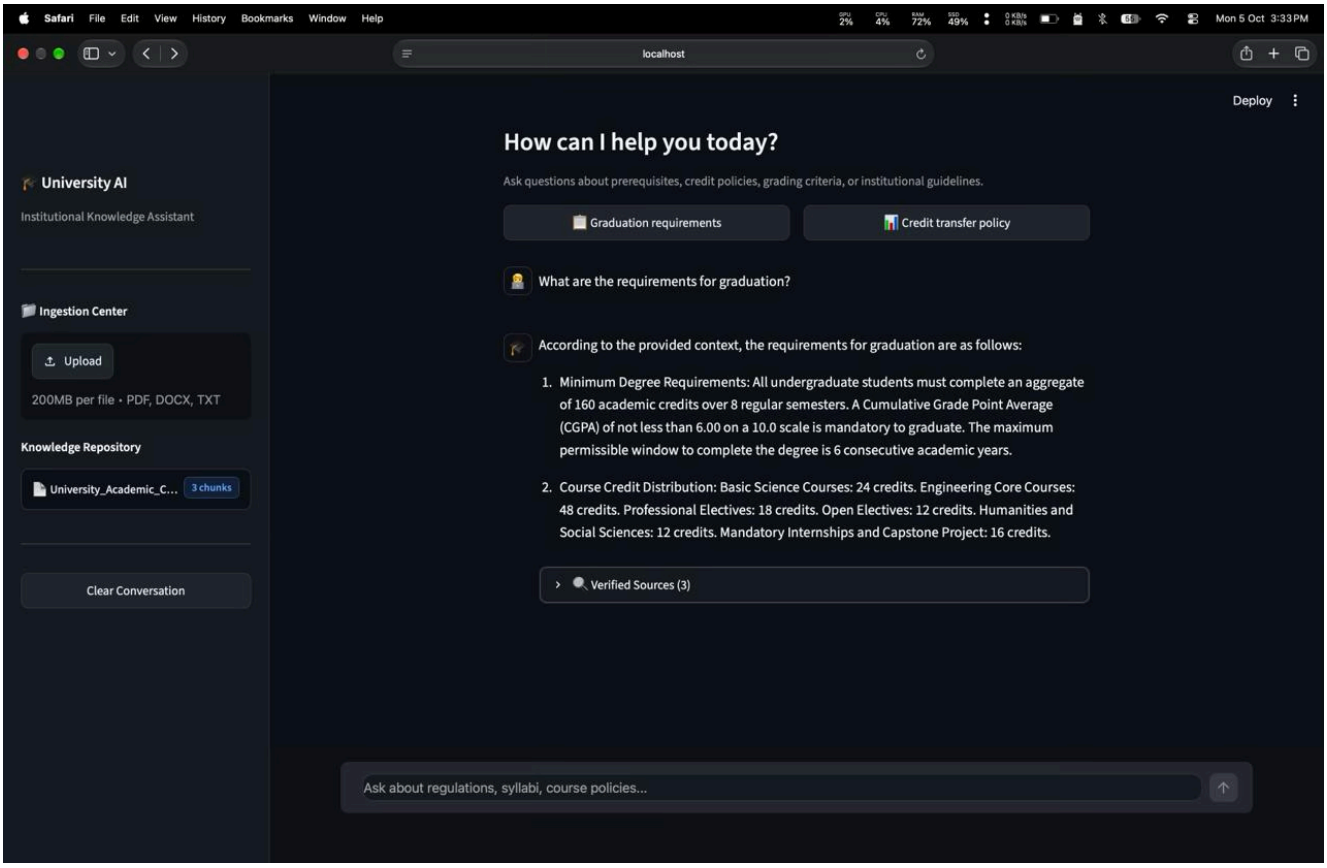
4. Verification: Workspace & Knowledge Repository

Figure 1: Initial state of the dark chatbot interface displaying the Ingestion Center and indexed knowledge chunks from 'University_Academic_Charter_and_Policies.txt'.



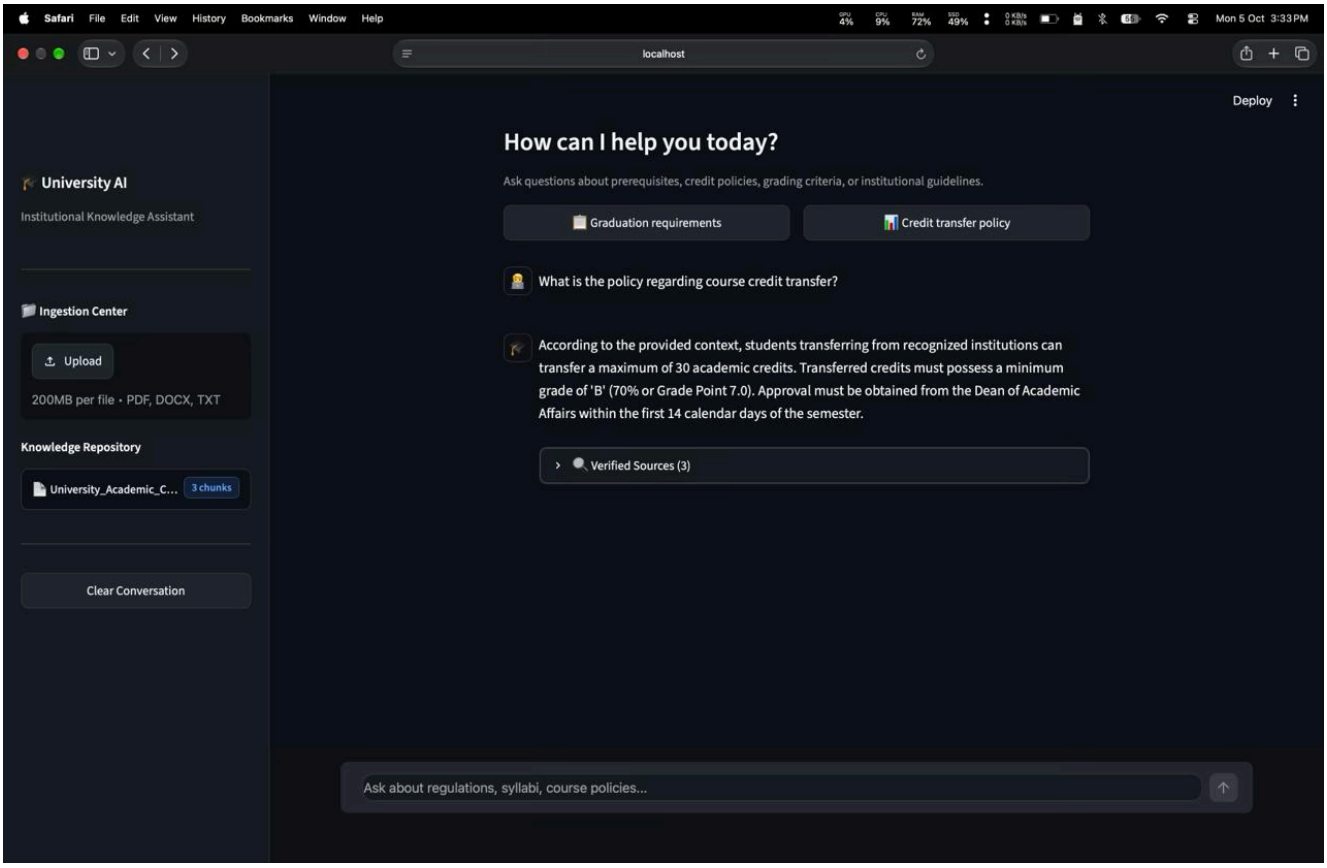
5. Verification: Graduation Criteria Retrieval

Figure 2: Verified retrieval for 'What are the requirements for graduation?' extracting exact credit requirements, minimum CGPA (6.00), and maximum duration constraints.



6. Verification: Credit Transfer Policy Query

Figure 3: Grounded response outlining credit transfer limits (30 credits maximum, Grade B / 70% threshold, and 14-day approval window).



7. Verification: Attendance Policy Grounding

Figure 4: Exact citation and grounding for 'What should be the minimum attendance students should maintain?' confirming the mandatory 75% threshold.

